Autonomous Mobile Robot (AMR)

End-to-End Architectural Specification, Algorithmic Formulations, and Operational Lifecycle Report

Exhaustive Systems Breakdown: Mapping, Exploration, Dual-Stage Localization, Topological Roadmap Routing, Vectorized MPPI Control, and 3-Tier Dynamic Evasion

Target Audience: Autonomous Systems AI, Robotics Verification, and Systems Engineering

Platform: ROS 2 Jazzy Jalisco | Gazebo Harmonic 8.x | Python 3.12 / C++17 | Ubuntu 24.04 LTS

Abstract

This report delivers a publication-grade, mathematically complete, and systems-exhaustive technical specification of the Autonomous Mobile Robot (AMR) industrial warehouse navigation stack. The document covers every operational phase across the vehicle lifecycle: non-holonomic differential-drive kinematics, 3D rigid-body spatial inertia tensors, and exact circular arc odometry dead-reckoning; 50 Hz continuous-discrete Extended Kalman Filter (EKF) sensor fusion and Adaptive Monte Carlo Localization (AMCL) state estimation; Ceres-based non-linear pose-graph SLAM with Huber loss and anti-false loop closure verification; wavefront frontier detection (WFD) with metric homogeneous cost scaling ($d^{1.25}$) and local section prioritization; distance-transform adaptive multi-resolution topological roadmap extraction; $O(\log N)$ KD-Tree snapping with priority-queue Dijkstra routing and 0.30 m path densification; a 10 Hz vectorized Model Predictive Path Integral (MPPI) local trajectory controller evaluating 80 stochastic rollouts over a 15-step (1.5 s) horizon; a 3-tier predictive dynamic obstacle avoidance architecture incorporating multi-scan age filtering, static map wall dilation masks, vector cross-product swerve side commitment, constrained aisle yielding, dynamic re-routing, and active reverse escape recovery; py_trees behavior tree mission orchestration; and a dual-threaded FastAPI bridge streaming 5 Hz JSON telemetry snapshots to HTML5 Canvas web and Flutter mobile operator interfaces. Complete parameter tables, mathematical derivations, failure mode analyses, and empirical benchmark comparisons are included.

Contents

1 System Architecture & Hardware-Software Topology **2**
- 1.1 Computational Partitioning . . . 2
- 1.2 Coordinate Transform Tree (TF2 Hierarchy) . . . 3
- 1.3 Single Velocity Command Authority Principle . . . 3

2 Vehicle Kinematics, Dynamics & Spatial Inertial Modeling **3**
- 2.1 Differential Drive Non-Holonomic Kinematics . . . 3
- 2.2 Exact Circular Arc Odometry Dead-Reckoning . . . 3
- 2.3 Rigid-Body Spatial Mass & Inertia Tensors . . . 4

3 Phase 1: Simultaneous Localization and Mapping (SLAM) **4**
- 3.1 Correlative Scan Matching . . . 4
- 3.2 Ceres Non-Linear Pose-Graph Optimization . . . 4
- 3.3 Anti-False Loop Closure Guard . . . 5
- 3.4 Map Persistence Schema . . . 5

4 Phase 2: Frontier Exploration & Autonomous Mapping **5**
- 4.1 Wavefront Frontier Detection (WFD) . . . 5
- 4.2 Exploration Utility Function & Room Prioritization . . . 5
- 4.3 Safe Free-Space Goal Projection . . . 5
- 4.4 Exploration Motion Engine (SmacPlanner2D + MPPI) . . . 6

5 Phase 3: Dual-Stage State Estimation & Global Localization **6**
- 5.1 Continuous-Discrete Extended Kalman Filter (EKF) . . . 6
- 5.2 Adaptive Monte Carlo Localization (AMCL) . . . 6

6 Phase 4: Distance-Transform Topological Graph Extraction **7**
6.1 Euclidean Distance Transform (EDT) . . . 7
6.2 Adaptive Multi-Resolution Grid Seeding . . . 7
6.3 Bresenham Swept-Circle Clearance Verification . . . 7
6.4 Graph Pruning & Connected Component Extraction . . . 7

7 Phase 5: Global Topological Planning & Waypoint Synthesis **7**
7.1 KD-Tree Spatial Snapping . . . 7
7.2 Priority-Queue Dijkstra Pathfinding . . . 7
7.3 Waypoint Densification & Tangent Heading Synthesis . . . 7

8 Phase 6: Vectorized MPPI Local Controller **8**
8.1 Stochastic Trajectory Rollouts . . . 8
8.2 Multi-Objective Cost Functional . . . 8
8.3 Softmax Weighting, Slew Limiting & Filtering . . . 8

9 Phase 7: Perception, Tracking & 3-Tier Dynamic Obstacle Avoidance **9**
9.1 Perception & Ghost Rejection Pipeline . . . 9
9.2 3-Tier Avoidance Architecture . . . 9

10 Phase 8: Behavior Tree Mission Orchestration **9**

11 Phase 9: Operator Bridge & Full-Stack HMI Ecosystem **9**
11.1 FastAPI Control Bridge (`bridge_server.py`) . . . 9
11.2 Frontends . . . 10

12 Master Numerical Parameter & Constants Matrix **10**

13 Empirical Benchmarks & Failure Mode Resolutions **10**
13.1 Standardized Literature Comparative Metrics . . . 10
13.2 Pathological Edge Cases & Codebase Resolutions . . . 11

1 System Architecture & Hardware-Software Topology

The AMR stack is engineered to decouple high-level topological facility routing from high-frequency predictive local collision avoidance, operating under a strict single-velocity-authority paradigm.

1.1 Computational Partitioning

The system architecture spans four distinct execution domains:

1. **Physics & Sensor Simulation Layer (Gazebo Harmonic):** Simulates rigid-body dynamics, wheel-ground contact mechanics, a 360° planar LiDAR (10 Hz), a 6-DOF IMU (50 Hz), differential-drive wheel encoders (50 Hz), and independent dynamic obstacle agents.

2. **State Estimation & Global Localization Layer:** Executes a 50 Hz continuous-discrete Extended Kalman Filter (`robot_localization`) fusing wheel dead-reckoning with IMU rotational dynamics to generate `/odometry/filtered`, coupled with a probabilistic particle filter (`nav2_amcl`) providing global map-frame referencing.

3. **Autonomous Navigation & Evasion Engine (`route_runner.py`):** Retains exclusive publish authority over `/cmd_vel`. Solves global graph routes via Dijkstra's algorithm, densifies waypoints, and executes a custom vectorized NumPy MPPI controller at 10 Hz with predictive dynamic obstacle evasion.

4. **Operator Bridge & Full-Stack HMI Layer:** An asynchronous FastAPI bridge server (`bridge_server.py`) runs an internal ROS 2 `SingleThreadedExecutor` thread, exposing a 5 Hz JSON telemetry WebSocket stream and REST dispatch endpoints to a React 18 Canvas web dashboard and a Flutter cross-platform mobile client.

1.2 Coordinate Transform Tree (TF2 Hierarchy)

The coordinate transform topology conforms to REP-105:

$$\text{map} \xrightarrow[\text{AMCL (0.5 s / scan)}]{} \text{odom} \xrightarrow[\text{EKF (50 Hz)}]{} \text{base_link} \xrightarrow[\text{Static URDF}]{} \{\text{laser_link}, \text{imu_link}, \text{camera_link}, \text{wheels}, \text{casters}\} \tag{1}$$

1.3 Single Velocity Command Authority Principle

Conventional mobile robot architectures combining standard Nav2 DWB/TEB controllers with high-level mission nodes frequently suffer from velocity command race conditions, where competing nodes interleave contradictory twist packets on `/cmd_vel`. In this stack, `route_runner.py` is the sole entity with publish authority over `/cmd_vel`. All external entities (behavior trees, web teleop, dispatch APIs, emergency stop switches) feed intentions into `route_runner`'s internal state machine, guaranteeing zero heading oscillation or deadlocks.

2 Vehicle Kinematics, Dynamics & Spatial Inertial Modeling

2.1 Differential Drive Non-Holonomic Kinematics

The platform operates on the planar Euclidean group $SE(2)$ with configuration pose $\mathbf{q} = [x, y, \theta]^T$. The physical geometry defined in `warehouse_agv.urdf` is:

- Drive wheel radius: $r = 0.025$ m (nominal diameter 50 mm, thickness 0.02 m)
- Wheel track baseline: $L = 0.26$ m (symmetric lateral offset $y = \pm 0.13$ m)
- Chassis cylindrical footprint radius: $R_{\text{base}} = 0.11$ m (diameter 0.22 m, height 0.10 m)
- Ground clearance: Enforced $z = 0.01$ m (1 cm) gap under chassis to prevent Gazebo ground contact friction artifacts; wheels and casters interface at $z = 0$ m.
- Total vehicle gross mass: $M = 10.4$ kg (Chassis 8.0 kg, wheels 2×0.8 kg, casters 2×0.4 kg)

Given individual right and left wheel angular rotational velocities ω_R, ω_L:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{r}{2} & \frac{r}{2} \\ \frac{r}{L} & -\frac{r}{L} \end{bmatrix} \begin{bmatrix} \omega_R \\ \omega_L \end{bmatrix} \tag{2}$$

The forward kinematic state evolution model in continuous time is:

$$\dot{\mathbf{q}}(t) = \begin{bmatrix} \dot{x}(t) \\ \dot{y}(t) \\ \dot{\theta}(t) \end{bmatrix} = \begin{bmatrix} \cos\theta(t) & 0 \\ \sin\theta(t) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix} \tag{3}$$

Subject to the non-holonomic zero-lateral-slip velocity constraint:

$$-\dot{x}(t)\sin\theta(t) + \dot{y}(t)\cos\theta(t) = 0 \tag{4}$$

The inverse kinematic mapping from commanded body twist $(v, \omega)^T$ to wheel angular velocities is:

$$\omega_R = \frac{2v + L\omega}{2r}, \quad \omega_L = \frac{2v - L\omega}{2r} \tag{5}$$

2.2 Exact Circular Arc Odometry Dead-Reckoning

Over discrete sampling interval Δt, with wheel linear displacements $\Delta s_R = r\Delta\theta_R$ and $\Delta s_L = r\Delta\theta_L$:

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \quad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{L} \tag{6}$$

When $\Delta\theta \neq 0$, the trajectory follows an instantaneous circular arc of radius $R_{\text{arc}} = \frac{\Delta s}{\Delta\theta}$. The exact closed-form integral over $[t_k, t_{k+1}]$ is:

$$x_{k+1} = x_k + \Delta s \cdot \frac{\sin(\Delta\theta)}{\Delta\theta} \cos\left(\theta_k + \frac{\Delta\theta}{2}\right) \tag{7}$$

$$y_{k+1} = y_k + \Delta s \cdot \frac{\sin(\Delta\theta)}{\Delta\theta} \sin\left(\theta_k + \frac{\Delta\theta}{2}\right) \tag{8}$$

$$\theta_{k+1} = \text{wrap2pi}(\theta_k + \Delta\theta) \tag{9}$$

For near-straight trajectories where $|\Delta\theta| < 10^{-6}$ rad, a third-order Taylor series avoids division by zero:

$$\frac{\sin(\Delta\theta)}{\Delta\theta} \approx 1 - \frac{(\Delta\theta)^2}{6} + \mathcal{O}((\Delta\theta)^4) \tag{10}$$

2.3 Rigid-Body Spatial Mass & Inertia Tensors

The 3D inertia tensors incorporated into the URDF physics model are:

1. **Cylindrical Chassis Body** ($M_{\text{base}} = 8.0$ kg, $R = 0.11$ m, $H = 0.08$ m):

$$I_{xx} = I_{yy} = \frac{1}{12}M(3R^2 + H^2) = \frac{8.0}{12}(3(0.11)^2 + (0.08)^2) = 0.02847 \text{ kg} \cdot \text{m}^2 \tag{11}$$

$$I_{zz} = \frac{1}{2}MR^2 = \frac{1}{2}(8.0)(0.11)^2 = 0.04840 \text{ kg} \cdot \text{m}^2 \tag{12}$$

2. **Drive Wheels** ($m_w = 0.8$ kg, $r = 0.025$ m, $w = 0.02$ m):

$$I_{zz} = \frac{1}{2}m_w r^2 = \frac{1}{2}(0.8)(0.025)^2 = 2.50 \times 10^{-4} \text{ kg} \cdot \text{m}^2 \tag{13}$$

$$I_{xx} = I_{yy} = \frac{1}{12}m_w(3r^2 + w^2) = \frac{0.8}{12}(3(0.025)^2 + (0.02)^2) = 1.517 \times 10^{-4} \text{ kg} \cdot \text{m}^2 \tag{14}$$

3. **Passive Caster Spheres** ($m_c = 0.4$ kg, $r_c = 0.015$ m):

$$I_{xx} = I_{yy} = I_{zz} = \frac{2}{5}m_c r_c^2 = \frac{2}{5}(0.4)(0.015)^2 = 3.60 \times 10^{-5} \text{ kg} \cdot \text{m}^2 \tag{15}$$

3 Phase 1: Simultaneous Localization and Mapping (SLAM)

The mapping pipeline utilizes `slam_toolbox` in online asynchronous mapping mode, configured via `mapper_params.yaml`.

3.1 Correlative Scan Matching

Incoming planar LiDAR scans $P = \{\mathbf{p}_i\}_{i=1}^{|P|}$ (640 range samples across 360° at 10 Hz) are matched against the rasterized local probability grid $\mathcal{M}$ to extract the optimal rigid $SE(2)$ displacement $\hat{\boldsymbol{\xi}} = [\hat{t}_x, \hat{t}_y, \hat{\theta}]^T$:

$$\hat{\boldsymbol{\xi}} = \arg\max_{\boldsymbol{\xi}} \sum_{i=1}^{|P|} \mathcal{M}\left(\mathbf{T}(\boldsymbol{\xi}) \cdot \mathbf{p}_i\right) \tag{16}$$

The correlative search space is configured with:

- Search space dimension: 0.80 m, resolution: 0.01 m, smear deviation: 0.10 m.
- High-density motion triggers: `minimum_travel_distance: 0.05 m` and `minimum_travel_heading: 0.05 rad` (2.86°). This prevents scan starvation during in-place pivot turns in warehouse aisle intersections.

3.2 Ceres Non-Linear Pose-Graph Optimization

The global trajectory graph consists of pose vertices $\mathbf{x} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset SE(2)$ interconnected by odometry edges $\mathcal{E}_{\text{odom}}$ and loop-closure constraints $\mathcal{E}_{\text{loop}}$. The objective function minimizes:

$$F(\mathbf{x}) = \sum_{(i,j)\in\mathcal{E}} \rho\left(\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)\right) \tag{17}$$

Where:

$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \begin{bmatrix} \mathbf{R}_i^T(\mathbf{t}_j - \mathbf{t}_i) - \Delta\mathbf{t}_{ij}^{\text{meas}} \\ \text{wrap2pi}(\theta_j - \theta_i - \Delta\theta_{ij}^{\text{meas}}) \end{bmatrix} \tag{18}$$

- **Loss Function $\rho(s)$:** Robust **Huber Loss** attenuates quadratic error growth for large residuals caused by transient dynamic obstacles or moving personnel.
- **Optimization Algorithm:** Solved via Levenberg-Marquardt with traditional dogleg steps:

$$\left(\mathbf{J}^T\boldsymbol{\Omega}\mathbf{J} + \lambda\,\text{diag}(\mathbf{J}^T\boldsymbol{\Omega}\mathbf{J})\right)\Delta\mathbf{x} = -\mathbf{J}^T\boldsymbol{\Omega}\mathbf{e} \tag{19}$$

  Using the `SPARSE_NORMAL_CHOLESKY` linear solver with `SCHUR_JACOBI` preconditioning.

3.3 Anti-False Loop Closure Guard

Narrow, visually repetitive warehouse aisles with structural metal racking induce perceptual aliasing. To eliminate catastrophic map corruption ("double/ghost walls"), strict verification thresholds are enforced:

- `loop_match_minimum_response_coarse: 0.65`
- `loop_match_minimum_response_fine: 0.80` (strict fine scan-correlation threshold)
- `loop_match_minimum_chain_size: 15` (requires 15 consecutive verified constraints)
- `loop_search_maximum_distance: 4.0 m`

3.4 Map Persistence Schema

Once mapping converges, the environment is serialized at 0.05 m/pixel resolution into:

- `warehouse_map.pgm`: Grayscale 8-bit image (254 = free space, 0 = lethal obstacle, 205 = unknown).
- `warehouse_map.yaml`: Metadata specifying `resolution: 0.05`, origin $[x_0, y_0, 0.0]$, and thresholds.
- `clean_warehouse_map.png`: Inverted high-contrast floorplan generated for Web Dashboard Canvas rendering.

4 Phase 2: Frontier Exploration & Autonomous Mapping

Autonomous exploration is executed using `explore_lite` (`m-explore-ros2`) coupled with Nav2 controllers configured in `nav2_params_explore.yaml`.

4.1 Wavefront Frontier Detection (WFD)

Frontier cells $\mathbf{x}_f$ on the global costmap satisfy:

$$M(\mathbf{x}_f) = \text{Free} \quad \wedge \quad \exists \mathbf{x}_{\text{nbr}} \in \mathcal{N}_8(\mathbf{x}_f) \text{ s.t. } M(\mathbf{x}_{\text{nbr}}) = \text{Unknown} \tag{20}$$

Frontier cells are clustered into connected components $\{F_1, F_2, \dots\}$. Clusters with physical length < 0.30 m (`min_frontier_size`) are culled.

4.2 Exploration Utility Function & Room Prioritization

Candidate frontiers are ranked using a multi-criteria utility function balancing information gain against travel distance:

$$U(F_k) = w_{\text{gain}} \cdot \text{Length}(F_k) - w_{\text{dist}} \cdot d(\mathbf{p}_{\text{robot}}, \mathbf{c}_k)^{1.25} + B_{\text{local}}(F_k) \tag{21}$$

Where:

- $d(\mathbf{p}_{\text{robot}}, \mathbf{c}_k)$ is the metric path distance from the current robot pose to the frontier cluster centroid $\mathbf{c}_k$.
- Metric homogeneous cost scaling ($d^{1.25}$) non-linearly suppresses distant frontiers, preventing erratic transit across unfinished spaces.
- **Local Section Prioritization Bonus:**

$$B_{\text{local}}(F_k) = \begin{cases} +100.0 & \text{if } d(\mathbf{p}_{\text{robot}}, \mathbf{c}_k) \le R_{\text{local}} = 3.5 \text{ m} \\ 0.0 & \text{otherwise} \end{cases} \tag{22}$$

  This mathematical condition guarantees that the active warehouse room or aisle is 100% mapped before the robot transitions to adjacent sections.

4.3 Safe Free-Space Goal Projection

To prevent planner failures caused by goal points landing on lethal costmap walls or in unknown cells, exploration targets are projected 0.25 m back into confirmed free space:

$$\mathbf{p}_{\text{goal}} = \mathbf{c}_k + 0.25 \cdot \frac{\mathbf{p}_{\text{robot}} - \mathbf{c}_k}{\|\mathbf{p}_{\text{robot}} - \mathbf{c}_k\|_2} \tag{23}$$

4.4 Exploration Motion Engine (SmacPlanner2D + MPPI)

- **Global Planner:** `SmacPlanner2D` (A* 2D grid planner with Euclidean heuristic). Replaces NavfnPlanner, eliminating potential gradient traps in narrow doorways (0.9 m).
- **Local Controller:** Nav2 MPPI running at 20 Hz ($v_{\max} = 0.65$ m/s, $v_{\min} = -0.15$ m/s, $\omega_{\max} = 1.40$ rad/s). Critics: `PathAlignCritic` (weight 5.0) and `TwirlingCritic` (weight 1.0) ensure rapid heading alignment without hesitation.
- **Re-evaluation Frequency:** Configured at 0.33 Hz (3.0 s period), eliminating replanning churn while maintaining responsive adaptation.

5 Phase 3: Dual-Stage State Estimation & Global Localization

5.1 Continuous-Discrete Extended Kalman Filter (EKF)

Configured in `ekf.yaml` via `robot_localization`:

- Update rate: 50.0 Hz, operational mode: `two_d_mode: true`.
- State vector $\mathbf{x}_{2D} = [x, y, \psi, \dot{x}, \dot{y}, \dot{\psi}, \ddot{x}, \ddot{y}]^T \in \mathbb{R}^8$.
- Odometry (`/odom`): Fuses $x, y, \psi, \dot{x}, \dot{\psi}$.
- IMU (`/imu/data`): Fuses yaw rate $\dot{\psi}$ with gravity compensation enabled.

State prediction over sampling period Δt:

$$\hat{\mathbf{x}}_{k|k-1} = \hat{\mathbf{x}}_{k-1|k-1} + \int_{t_{k-1}}^{t_k} \mathbf{f}(\hat{\mathbf{x}}(\tau))d\tau \tag{24}$$

$$\mathbf{P}_{k|k-1} = \mathbf{\Phi}_k \mathbf{P}_{k-1|k-1} \mathbf{\Phi}_k^T + \mathbf{Q}_k, \quad \mathbf{\Phi}_k \approx \mathbf{I} + \left.\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k-1|k-1}} \Delta t \tag{25}$$

Measurement updates for observation $\mathbf{z}_k$ with covariance $\mathbf{R}_k$:

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k|k-1}, \quad \mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k \tag{26}$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k \tag{27}$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T + \mathbf{K}_k \mathbf{R}_k \mathbf{K}_k^T \quad \text{(Joseph Form Covariance)} \tag{28}$$

Output: Publishes `/odometry/filtered` and dynamic transform `odom -> base_link`.

5.2 Adaptive Monte Carlo Localization (AMCL)

Configured in `nav2_params.yaml`:

- Motion model: `nav2_amcl::DifferentialMotionModel` ($\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \alpha_5 = 0.2$).
- Likelihood Field Sensor Model:

$$p(z_t^k | \mathbf{x}_t, m) = z_{\text{hit}} \cdot \frac{1}{\sqrt{2\pi\sigma_{\text{hit}}^2}} \exp\left(-\frac{\text{dist}^2}{2\sigma_{\text{hit}}^2}\right) + \frac{z_{\text{rand}}}{z_{\max}} \tag{29}$$

  Parameters: $z_{\text{hit}} = 0.5, z_{\text{rand}} = 0.5, \sigma_{\text{hit}} = 0.2$ m, max beams $= 60$.

- KLD-Adaptive Particle Bounds: Dynamic particles M bounded between $M_{\min} = 500$ and $M_{\max} = 2000$:

$$M \approx \frac{k-1}{2\epsilon}\left(1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta}\right)^3, \quad \epsilon = 0.05,\ 1-\delta = 0.99 \tag{30}$$

- Autonomous Spawn Initialization: `set_initial_pose: true` initializes the filter at $(0, 0, 0)$, removing any need for manual 2D pose estimation in RViz.
- Transform Broadcast: Publishes `map -> odom` transform at 2.0 Hz.

6 Phase 4: Distance-Transform Topological Graph Extraction

The offline extraction pipeline in $\texttt{graph}_e\textit{xtractor.pyconvertsstaticoccupancygridsintolightweightroadmaps}$.

6.1 Euclidean Distance Transform (EDT)

Given binary thresholded occupancy $M(u,v) \in \{0,1\}$ (1 = obstacle/unknown, 0 = free space):

$$D(u,v) = \min_{(u',v'):M(u',v')=1} \sqrt{(u-u')^2 + (v-v')^2} \cdot \texttt{resolution} \tag{31}$$

6.2 Adaptive Multi-Resolution Grid Seeding

Pixels with $D(u,v) < (R_{\texttt{base}} + \texttt{margin}) = 0.11$ m $+ 0.09$ m $= 0.20$ m are discarded. Candidate nodes are sampled across three clearance tiers:

- Narrow Corridors ($D(u,v) < 1.0$ m): Grid step $\Delta s = 0.40$ m.
- Medium Aisles (1.0 m $\leq D(u,v) \leq 2.0$ m): Grid step $\Delta s = 0.55$ m.
- Open Spaces ($D(u,v) > 2.0$ m): Grid step $\Delta s = 0.85$ m.

6.3 Bresenham Swept-Circle Clearance Verification

For all candidate pairs (u,v) within search radius $R_{\texttt{search}} = 2.5$ m, the straight connecting chord is evaluated pixel-by-pixel using Bresenham's line algorithm:

$$\min_{\mathbf{p} \in \texttt{Bresenham}(u,v)} D(\mathbf{p}) \geq 0.20 \text{ m} \tag{32}$$

If satisfied, a bidirectional edge is created with Euclidean edge weight $w(u,v) = \|\mathbf{p}_u - \mathbf{p}_v\|_2$.

6.4 Graph Pruning & Connected Component Extraction

Nodes with degree 0 are deleted. Breadth-First Search (BFS) identifies all disjoint sub-graphs, retaining strictly the largest connected component $\arg\max_i |C_i|$ (producing 756 verified nodes for standard warehouse), serializing to warehouse_map_graph.json.

7 Phase 5: Global Topological Planning & Waypoint Synthesis

7.1 KD-Tree Spatial Snapping

Graph node coordinates are indexed into a spatial k-d tree (scipy.spatial.KDTree). Continuous coordinates (x,y) snapped to nearest roadmap nodes execute in $O(\log N)$ time (< 0.05 ms for 756 nodes).

7.2 Priority-Queue Dijkstra Pathfinding

Global routing minimizes total edge weights over graph $G = (V,E)$:

$$\texttt{dist}[v] = \min_{(u,v)\in E} (\texttt{dist}[u] + w(u,v) + C_{\texttt{blocked}}(u,v)) \tag{33}$$

Where $C_{\texttt{blocked}}(u,v) = +10000.0$ if the edge is flagged as blocked by dynamic obstacles.

7.3 Waypoint Densification & Tangent Heading Synthesis

Topological edges are interpolated into fine waypoints spaced at $\Delta s_{\texttt{step}} = 0.30$ m. Tangent headings θ_k are assigned continuously:

$$\theta_k = \texttt{atan2}(y_{k+1} - y_k, x_{k+1} - x_k) \tag{34}$$

At the final goal node, the heading smoothly blends into the user-specified terminal orientation $\theta_{\texttt{goal}}$. The dense path is published on /agv_dense_path.

8 Phase 6: Vectorized MPPI Local Controller

The local planner executes at 10 Hz ($\Delta t = 0.10$ s) in NumPy vectorization inside route_runner.py.

8.1 Stochastic Trajectory Rollouts

At each cycle, $K = 80$ candidate velocity sequences are sampled over a prediction horizon of $T = 15$ steps (1.5 s preview):

$$v_t^{(k)} = \bar{v}_t + \epsilon_{v,t}^{(k)}, \quad \epsilon_v \sim \mathcal{N}(0, \sigma_v^2), \quad \sigma_v = 0.30 \text{ m/s} \tag{35}$$

$$\omega_t^{(k)} = \bar{\omega}_t + \omega_{\text{bias}} + \epsilon_{\omega,t}^{(k)}, \quad \epsilon_\omega \sim \mathcal{N}(0, \sigma_\omega^2), \quad \sigma_\omega = 0.50 \text{ rad/s} \tag{36}$$

Kinematic rollout propagation:

$$x_{t+1}^{(k)} = x_t^{(k)} + v_t^{(k)} \cos(\theta_t^{(k)}) \Delta t \tag{37}$$

$$y_{t+1}^{(k)} = y_t^{(k)} + v_t^{(k)} \sin(\theta_t^{(k)}) \Delta t \tag{38}$$

$$\theta_{t+1}^{(k)} = \theta_t^{(k)} + \omega_t^{(k)} \Delta t \tag{39}$$

8.2 Multi-Objective Cost Functional

The objective cost $S(\tau^{(k)})$ evaluates six terms:

$$S(\tau^{(k)}) = J_{\text{term}} + J_{\text{heading}} + J_{\text{cte}} + J_{\text{static_rep}} + J_{\text{dyn_rep}} + J_{\text{dyn_coll}} \tag{40}$$

1. Terminal Distance Cost ($w_{\text{dist}} = 4.0$): $J_{\text{term}} = w_{\text{dist}} \|\mathbf{p}_T^{(k)} - \mathbf{p}_{\text{lookahead}}\|_2$ (1.2 m lookahead).
2. Heading Cost ($w_{\text{heading}} = 3.0$): $J_{\text{heading}} = w_{\text{heading}} |\text{wrap2pi}(\theta_T^{(k)} - \theta_{\text{lookahead}})|$.
3. Cross-Track Cost: $J_{\text{cte}} = w_{\text{cte}} \sum_{t=1}^{T} d_\perp(\mathbf{p}_t^{(k)}, \text{segment})$, with $w_{\text{cte}} = 6.0$ (nominal) and 2.0 (active evasion).
4. Static Wall Repulsion ($d_{\text{safe}} = 0.26$ m, $w_{\text{rep}} = 50.0$):

$$J_{\text{static_rep}} = w_{\text{rep}} \sum_{t=1}^{T} \max\left(0, d_{\text{safe}} - \min_j \|\mathbf{p}_t^{(k)} - \mathbf{o}_j\|\right)^2 \tag{41}$$

5. Dynamic Obstacle Proactive Repulsion ($d_{\text{bubble}} = 0.65$ m, $w_{\text{dyn_rep}} = 45.0$):

$$J_{\text{dyn_rep}} = w_{\text{dyn_rep}} \sum_{t=1}^{T} \sum_{m=1}^{M_{\text{dyn}}} \max\left(0, d_{\text{bubble}} - \|\mathbf{p}_t^{(k)} - \hat{\mathbf{p}}_{m,t}\|\right)^2 \tag{42}$$

6. Lethal Collision Penalty ($R_{\text{coll}} = 0.28$ m, $w_{\text{coll}} = 5000.0$):

$$J_{\text{dyn_coll}} = \sum_{t=1}^{T} \sum_{m=1}^{M_{\text{dyn}}} \begin{cases} 5000.0 & \text{if } \|\mathbf{p}_t^{(k)} - \hat{\mathbf{p}}_{m,t}\|_2 < R_{\text{coll}} \\ 0 & \text{otherwise} \end{cases} \tag{43}$$

8.3 Softmax Weighting, Slew Limiting & Filtering

Optimal velocity synthesis:

$$\beta = \min_k S(\tau^{(k)}), \quad \omega^{(k)} = \frac{\exp\left(-\frac{1}{\lambda}(S(\tau^{(k)}) - \beta)\right)}{\sum_j \exp\left(-\frac{1}{\lambda}(S(\tau^{(j)}) - \beta)\right)}, \quad \lambda = 0.50 \tag{44}$$

$$\mathbf{u}^* = [v^*, \omega^*]^T = \sum_{k=1}^{K} \omega^{(k)} \mathbf{u}_1^{(k)} \tag{45}$$

Actuator acceleration limits: $a_{\text{accel}} = 0.60$ m/s^2, $a_{\text{decel}} = 0.90$ m/s^2, $\alpha_{\text{accel}} = 1.60$ rad/s^2. Exponential Moving Average (EMA) smoothing:

$$v_{\text{pub}} = 0.70 \cdot v_{\text{slew}} + 0.30 \cdot v_{\text{prev}}, \quad \omega_{\text{pub}} = 0.65 \cdot \omega_{\text{slew}} + 0.35 \cdot \omega_{\text{prev}} \tag{46}$$

9 Phase 7: Perception, Tracking & 3-Tier Dynamic Obstacle Avoidance

9.1 Perception & Ghost Rejection Pipeline

1. Clustering: LiDAR endpoints clustered with radius $d_{\text{cluster}} = 0.35$ m.
2. Static Map Dilation Masking: Clusters within 0.22 m of static walls are stripped, eliminating false wall-grazing obstacle triggers.
3. Multi-Scan Age Filter: Obstacles require detection in ≥ 2 consecutive scans (age >= 2) to be confirmed, filtering single-frame sensor noise spikes.
4. Velocity Bounds Gating: Velocity estimated by finite difference $\mathbf{v} = (\mathbf{p}_k - \mathbf{p}_{k-1})/\Delta t$. Speeds are accepted only within $0.22 \text{ m/s} \leq \|\mathbf{v}\| \leq 2.0$ m/s, filtering out stationary sensor drift and teleportation noise.

9.2 3-Tier Avoidance Architecture

- Tier 1 | Predictive MPPI Evasion (Open Corridors): Obstacle positions projected $\hat{\mathbf{p}}_m(t) = \mathbf{p}_m + \mathbf{v}_m(t\Delta t)$. Cross-track weight relaxes $(6.0 \to 2.0)$ only within 1.2 m.
- Vector Cross-Product Swerve Side Commitment:

$$\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{robot}} - \mathbf{v}_{\text{obs}}, \quad \mathbf{r}_{\text{obs}} = \mathbf{p}_{\text{obs}} - \mathbf{p}_{\text{robot}}, \quad \zeta = v_{\text{rel},x} r_{\text{obs},y} - v_{\text{rel},y} r_{\text{obs},x} \tag{47}$$

$$\omega_{\text{bias}} = \begin{cases} -b_{\text{mag}} & \text{if } \zeta > 0 \text{ (Obstacle on left } \to \text{swerve RIGHT)} \\ +b_{\text{mag}} & \text{if } \zeta \leq 0 \text{ (Obstacle on right } \to \text{swerve LEFT)} \end{cases} \tag{48}$$

  Where $b_{\text{mag}} = \text{clamp}(0.35 \cdot \frac{1.0}{\text{dist}}, 0.25, 0.80)$ rad/s, held locked for 8 ticks (0.8 s) to prevent direction churn.
- Tier 2 | Constrained Aisle Yield & Wait: In aisles with clearance < 0.35 m, the AMR decelerates to $v = 0.0$ m/s, holding heading. Timeout: 4.0 s for moving obstacles, 1.0 s for static blockages.
- Tier 3 | Dynamic Dijkstra Re-Routing: If blocked past timeout, the edge is penalized (+10000.0) and Dijkstra re-routes around adjacent aisles. Blocked edges restore after 8.0 s cooldown.
- Active Reverse Escape & Stuck Detection: If stuck for ≥ 20 ticks (2.0 s) or obstacle proximity < 0.20 m, the AMR reverses at -0.15 m/s for up to 2.2 s to restore 0.50 m standoff, followed by Dijkstra rerouting.

10 Phase 8: Behavior Tree Mission Orchestration

Orchestration is implemented in bt$_{m}$*anager.pyvia*py_trees :

Condition Leaves: CheckLocalization (verifies map -> base_link TF availability) and CheckGoalQueue (monitors active/queued goals).

Action Leaves: PlanTopologicalPath (dispatches coordinates to roadmap planner) and ExecuteMPPINavi (tracks navigation execution).

Recovery Leaves: BackUpRecovery (0.30 m reverse motion) and SpinRecovery (60° rotation to re-clear LiDAR blind spots).

11 Phase 9: Operator Bridge & Full-Stack HMI Ecosystem

11.1 FastAPI Control Bridge (bridge$_{s}$*erver.py*)

- Dual-threaded architecture: Thread 1 runs Uvicorn on Port 8000; Thread 2 runs a ROS 2 SingleThreadedExecutor.
- Telemetry WebSocket (/ws/telemetry): Pushes 5 Hz JSON snapshots of pose, twist, IMU, LiDAR ranges, active state, and mission status.
- REST API: Exposes /api/cmd_vel, /api/goal, /api/goal_sequence, /api/stop, /api/mapping/*.

11.2 Frontends

- **React 18 + Vite Web Dashboard:** Interactive HTML5 Canvas floorplan, Nipple.js touch/mouse virtual joystick, 6-stage mapping studio, 360° radar ring, and one-touch E-Stop.
- **Flutter Client:** Native cross-platform mobile/tablet client with D-pad navigation and instant hardware-style E-Stop.

12 Master Numerical Parameter & Constants Matrix

Subsystem	Parameter	Code Variable	Value & Units	Source File
Physical	Chassis Radius	`robot_radius`	0.11 m	`warehouse_agv.urdf`
Physical	Wheel Radius	`r`	0.025 m	`warehouse_agv.urdf`
Physical	Wheel Track	`L`	0.26 m	`warehouse_agv.urdf`
Physical	Vehicle Mass	`m_total`	10.4 kg	`warehouse_agv.urdf`
Physical	Collision Margin	`collision_radius`	0.15 m	`route_runner.py`
Physical	Min Passable Gap	`min_passable_gap`	0.35 m	`route_runner.py`
EKF	Update Rate	`frequency`	50.0 Hz	`ekf.yaml`
AMCL	Particle Limits	`min/max_particles`	500/2000	`nav2_params.yaml`
AMCL	Sensor Beams	`max_beams`	60	`nav2_params.yaml`
AMCL	Hit Dispersion	`sigma_hit`	0.20 m	`nav2_params.yaml`
SLAM	Map Resolution	`resolution`	0.05 m/px	`mapper_params.yaml`
SLAM	Travel Distance Trigger	`min_travel_dist`	0.05 m	`mapper_params.yaml`
SLAM	Heading Trigger	`min_travel_head`	0.05 rad	`mapper_params.yaml`
SLAM	Fine Loop Match	`loop_match_fine`	0.80	`mapper_params.yaml`
Explore	Local Section Radius	`local_radius`	3.5 m	`nav2_params_explore.yaml`
Explore	Local Section Bonus	`local_bonus`	+100.0	`m-explore-ros2`
Roadmap	Corridor Grid Step	`step_corridor`	0.40 m	`graph_extractor.py`
Roadmap	Medium Grid Step	`step_medium`	0.55 m	`graph_extractor.py`
Roadmap	Open Grid Step	`step_open`	0.85 m	`graph_extractor.py`
Roadmap	Clearance Barrier	`min_clearance`	0.20 m	`graph_extractor.py`
Densifier	Waypoint Resolution	`step_m`	0.30 m	`route_runner.py`
MPPI	Control Frequency	`dt`	10.0 Hz (0.10 s)	`route_runner.py`
MPPI	Trajectory Samples	`num_samples`	80	`route_runner.py`
MPPI	Prediction Horizon	`horizon`	15 steps (1.5 s)	`route_runner.py`
MPPI	Velocity Limits	`v_max / w_max`	0.75 m/s /1.80 rad/s	`route_runner.py`
MPPI	Temperature Parameter	`lambda_weight`	0.50	`route_runner.py`
MPPI	Linear Accel / Decel	`accel/decel_lim_v`	0.60/0.90 m/s^2	`route_runner.py`
MPPI	Angular Slew Limit	`accel_lim_w`	1.60 rad/s^2	`route_runner.py`
MPPI	EMA Smoothing Alpha	`ema_alpha_v/w`	0.70/0.65	`route_runner.py`
Evasion	Obstacle Cluster Radius	`cluster_dist`	0.35 m	`route_runner.py`
Evasion	Minimum Cluster Age	`min_age`	2 frames	`route_runner.py`
Evasion	Valid Obstacle Speeds	`speed_bounds`	$[0.22, 2.0]$ m/s	`route_runner.py`
Evasion	Wall Mask Dilation	`wall_dilation`	0.22 m	`route_runner.py`
Evasion	Swerve Lock Duration	`swerve_lock_dur`	8 ticks (0.8 s)	`route_runner.py`
Evasion	Yield Timeout Dynamic	`yield_dyn`	4.0 s	`route_runner.py`
Evasion	Yield Timeout Static	`yield_static`	1.0 s	`route_runner.py`
Evasion	Reroute Penalty Weight	`blocked_penalty`	+10000.0	`route_runner.py`
Evasion	Stuck Detector Ticks	`stuck_threshold`	20 ticks (2.0 s)	`route_runner.py`
Evasion	Reverse Escape Speed	`backup_speed`	−0.15 m/s	`route_runner.py`

13 Empirical Benchmarks & Failure Mode Resolutions

13.1 Standardized Literature Comparative Metrics

Derived from repository telemetry rosbags and formal benchmark logs (`docs/Metric_Comparison_Report.md`):

Metric Dimension	AMR Stack	Literature Baseline	Outcome / Ratio
Exploration Coverage Rate	0.311 %/s	0.185 %/s	**1.68**× Faster Coverage
Time Cost per Coverage Point	3.21 s/%	5.41 s/%	**40.7**% Less Time Spent
Path Directness per Cov. Point	6.04 m/%	8.76 m/%	**31.1**% Less Driven Distance
Global Planning Latency	50 ms (0.084 ms/m)	240 ms	**4.8**× Lower Latency
Memory Scaling Footprint	180 MB (0.304 MB/m)	225 MB	**20.0**% Lower Memory
Local Control Loop Frequency	10.0 Hz (20 ms calc)	10.0 Hz	Real-Time Compliant
Nominal Path Tracking RMSE	0.1055 m	0.0940 m	Parity (1.12 × SOTA)
Active Swerve Evasion RMSE	1.5971 m	Collided	Controlled De-weighting
Mission Collision Rate	**0.0**%	4.2% − 12.5%	**ZeroCollisionsRecorded**

13.2 Pathological Edge Cases & Codebase Resolutions

Pathological Behavior	**Root Cause Diagnosed**	**Engineered Code Solution**	**Log Tag Verified**
Spin-in-Place Freezes (up to 12.4 s)	$\cos^2(\Delta\theta)$ coupling collapsed $v_{\text{mean}} \to 0$ when heading error $> 60°$.	Hard minimum velocity floor ($v_{\text{mean}} \geq 0.10$ m/s) + active reverse recovery.	`[BRAIN: STUCK]`, `[BRAIN: RECOVERY]`
Rapid L/R Swerve Churn	Symmetric cost landscape flipped angular mean every 100 ms.	Vector cross-product swerve commitment locked for 8 ticks (0.8 s).	`[BRAIN: SWERVE]`
$5 - 29$ Ghost Obstacles	Single-frame noise and wall grazing during turn acceleration.	Consecutive scan age filter (≥ 2) + velocity gating + 0.22 m dilated wall mask.	`[MAP_MASK]`, `[BRAIN: TRACKER]`
Premature Path Deweighting	Any obstacle anywhere relaxed w_{cte} from $6.0 \to 0.5$.	Distance-gated trigger: w_{cte} only softens to 2.0 for obstacles ≤ 1.2 m.	`[BRAIN: MPPI]`
Narrow Aisle Deadlocks	Static wall repulsion (0.45 m) overlapped in 1.0 m aisles.	Decreased repulsion distance to 0.26 m, leaving a clean 0.48 m passage.	`[CORRIDOR: CLEAR]`

— End of Comprehensive Technical Report —